单例模式实现逻辑层设计

Posted on 2023-06-05 09:14:05 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2QIUQUVxmxquygX6trkUqZZMD4x)

优雅退出

服务器优雅退出一直是服务器设计必须考虑的一个方向，意在能通过捕获信号使服务器安全退出。我们可以通过asio提供的信号机制绑定回调函数即可实现优雅退出。在主函数中我们添加

```
1. int main()
2. {
3.     try {
4.         boost::asio::io_context  io_context;
5.         boost::asio::signal_set signals(io_context, SIGINT, SIGTERM);
6.         signals.async_wait([&io_context](auto, auto) {
7.             io_context.stop();
8.             });
9.         CServer s(io_context, 10086);
10.        io_context.run();
11.    }
12.    catch (std::exception& e) {
13.        std::cerr << "Exception: " << e.what() << endl;
14.    }
15. }
```

利用signal_set 定义了一系列信号合集，并且绑定了一个匿名函数，匿名函数捕获了io_context的引用，并且函数中设置了停止操作，也就是说当捕获到SIGINT,SIGTERM等信号时，会调用 `io_context.stop` 。

单例模板类

接下来我们实现一个单例模板类，因为服务器的逻辑处理需要单例模式，后期可能还会有一些模块的设计也需要单例模式，所以先实现一个单例模板类，然后其他想实现单例类只需要

继承这个模板类即可。

```
#include <memory>
#include <mutex>
#include <iostream>
using namespace std;
template <typename T>
class Singleton {
protected:
    Singleton() = default;
    Singleton(const Singleton<T>&) = delete;
    Singleton& operator=(const Singleton<T>& st) = delete;

    static std::shared_ptr<T> _instance;
public:
    static std::shared_ptr<T> GetInstance() {
        static std::once_flag s_flag;
        std::call_once(s_flag, [&]() {
            _instance = shared_ptr<T>(new T);
            });

        return _instance;
    }
    void PrintAddress() {
        std::cout << _instance.get() << endl;
    }
    ~Singleton() {
        std::cout << "this is singleton destruct" << std::endl;
    }
};

template <typename T>
std::shared_ptr<T> Singleton<T>::_instance = nullptr;
```

单例模式模板类将无参构造，拷贝构造，拷贝赋值都设定为protected属性，其他的类无法访问，其实也可以设置为私有属性。析构函数设置为公有的，其实设置为私有的更合理一点。

`Singleton` 有一个static类型的属性 `_instance` ，它是我们实际要开辟类型的智能指针类型。

`s_flag` 是函数 `GetInstance` 内的局部静态变量，该变量在函数 `GetInstance` 第一次调用时被初始化。以后无论调用多少次 `GetInstance` `s_flag` 都不会被重复初始化，而且 `s_flag` 存在静态区，会随着进程结束而自动释放。

`call_once` 只会调用一次，而且是线程安全的， 其内部的原理就是调用该函数时加锁，然后

设置s_flag内部的标记，设置为已经初始化，执行lambda表达式逻辑初始化智能指针，然后解锁。第二次调用 `GetInstance` 内部还会调用 `call_once`，只是 `call_once` 判断 `s_flag` 已经被初始化了就不执行初始化智能指针的操作了。

LogicSystem单例类

我们实现逻辑系统的单例类，继承自 `Singleton<LogicSystem>` ，这样LogicSystem的构造函数和拷贝构造函数就都变为私有的了，因为基类的构造函数和拷贝构造函数都是私有的。另外LogicSystem也用了基类的成员 `_instance` 和 `GetInstance` 函数。从而达到单例效果。

```
1. typedef  function<void(shared_ptr<CSession>, short msg_id, string msg_data)> FunCallBack;
2. class LogicSystem:public Singleton<LogicSystem>
3. {
4.     friend class Singleton<LogicSystem>;
5. public:
6.     ~LogicSystem();
7.     void PostMsgToQue(shared_ptr < LogicNode> msg);
8. private:
9.     LogicSystem();
10.    void DealMsg();
11.    void RegisterCallBacks();
12.    void HelloWordCallBack(shared_ptr<CSession>, short msg_id, string msg_data);
13.    std::thread _worker_thread;
14.    std::queue<shared_ptr<LogicNode>> _msg_que;
15.    std::mutex _mutex;
16.    std::condition_variable _consume;
17.    bool _b_stop;
18.    std::map<short, FunCallBack> _fun_callbacks;
19. };
```

1 `FunCallBack` 为要注册的回调函数类型，其参数为绘画类智能指针，消息id，以及消息内容。

2 `_msg_que` 为逻辑队列

3 `_mutex` 为保证逻辑队列安全的互斥量

4 `_consume` 表示消费者条件变量，用来控制当逻辑队列为空时保证线程暂时挂起等待，不要干扰其他线程。

5 `_fun_callbacks` 表示回调函数的map，根据id查找对应的逻辑处理函数。

6 `_worker_thread` 表示工作线程，用来从逻辑队列中取数据并执行回调函数。

7 `_b_stop` 表示收到外部的停止信号，逻辑类要中止工作线程并优雅退出。

`LogicNode` 定义在CSession.h中

```
1. class LogicNode {
2.     friend class LogicSystem;
3. public:
4.     LogicNode(shared_ptr<CSession>, shared_ptr<RecvNode>);
5. private:
6.     shared_ptr<CSession> _session;
7.     shared_ptr<RecvNode> _recvnode;
8. };
```

其包含算了会话类的智能指针，主要是为了实现伪闭包，防止session被释放。
其次包含了接收消息的节点类的智能指针。
实现如下

```
1. LogicNode::LogicNode(shared_ptr<CSession>  session,
2.     shared_ptr<RecvNode> recvnode):_session(session),_recvnode(recvnode) {
3.
4. }
```

`LogicSystem` 的构造函数如下

```
1. LogicSystem::LogicSystem():_b_stop(false){
2.     RegisterCallBacks();
3.     _worker_thread = std::thread (&LogicSystem::DealMsg, this);
4. }
```

构造函数中将停止信息初始化为false，注册消息处理函数并且启动了一个工作线程，工作线程执行DealMsg逻辑。
注册消息处理函数的逻辑如下

```
void LogicSystem::RegisterCallBacks() {
    _fun_callbacks[MSG_HELLO_WORD] = std::bind(&LogicSystem::HelloWordCallBack, this,
        placeholders::_1, placeholders::_2, placeholders::_3);
}
```

`MSG_HELLO_WORD` 定义在const.h中

```
enum MSG_IDS {
    MSG_HELLO_WORD = 1001
};
```

`MSG_HELLO_WORD` 表示消息id， `HelloWordCallBack` 为对应的回调处理函数

```
void LogicSystem::HelloWordCallBack(shared_ptr<CSession> session, short msg_id, string msg_data) {
    Json::Reader reader;
    Json::Value root;
    reader.parse(msg_data, root);
    std::cout << "recevie msg id  is " << root["id"].asInt() << " msg data is "
        << root["data"].asString() << endl;
    root["data"] = "server has received msg, msg data is " + root["data"].asString();
    std::string return_str = root.toStyledString();
    session->Send(return_str, root["id"].asInt());
}
```

在 `HelloWordCallBack` 里我们根据消息id和收到的消息，做了相应的处理并且回应给客户端。

工作线程的处理函数 `DealMsg` 逻辑

```
void LogicSystem::DealMsg() {
    for (;;) {
        std::unique_lock<std::mutex> unique_lk(_mutex);
        //判断队列为空则用条件变量阻塞等待，并释放锁
        while (_msg_que.empty() && !_b_stop) {
            _consume.wait(unique_lk);
        }

        //判断是否为关闭状态，把所有逻辑执行完后则退出循环
        if (_b_stop ) {
            while (!_msg_que.empty()) {
                auto msg_node = _msg_que.front();
                cout << "recv_msg id  is " << msg_node->_recvnode->_msg_id << endl;
                auto call_back_iter = _fun_callbacks.find(msg_node->_recvnode->_msg_id);
                if (call_back_iter == _fun_callbacks.end()) {
                    _msg_que.pop();
                    continue;
                }
                call_back_iter->second(msg_node->_session, msg_node->_recvnode->_msg_id,
                    std::string(msg_node->_recvnode->_data, msg_node->_recvnode->_cur_len));
                _msg_que.pop();
            }
            break;
        }

        //如果没有停服，且说明队列中有数据
        auto msg_node = _msg_que.front();
        cout << "recv_msg id  is " << msg_node->_recvnode->_msg_id << endl;
        auto call_back_iter = _fun_callbacks.find(msg_node->_recvnode->_msg_id);
        if (call_back_iter == _fun_callbacks.end()) {
            _msg_que.pop();
            continue;
        }
        call_back_iter->second(msg_node->_session, msg_node->_recvnode->_msg_id,
            std::string(msg_node->_recvnode->_data, msg_node->_recvnode->_cur_len));
        _msg_que.pop();
    }
}
```

1 `DealMsg` 逻辑中初始化了一个unique_lock，主要是用来控制队列安全，并且配合条件变量可以随时解锁。lock_guard不具备解锁功能，所以此处用unique_lock。

2 我们判断队列为空，并且不是停止状态，就挂起线程。否则继续执行之后的逻辑，如果_b_stop为true，说明处于停服状态，则将队列中未处理的消息全部处理完然后退出循环。如果_b_stop未false，则说明没有停服，是consumer发送的激活信号激活了线程，则继续取队列中的数据处理。

LogicSystem的析构函数需要等待工作线程处理完再退出，但是工作线程可能处于挂起状态，所以要发送一个激活信号唤醒工作线程。并且将_b_stop标记设置为true。

```
1. LogicSystem::~LogicSystem(){
2.     _b_stop = true;
3.     _consume.notify_one();
4.     _worker_thread.join();
5. }
```

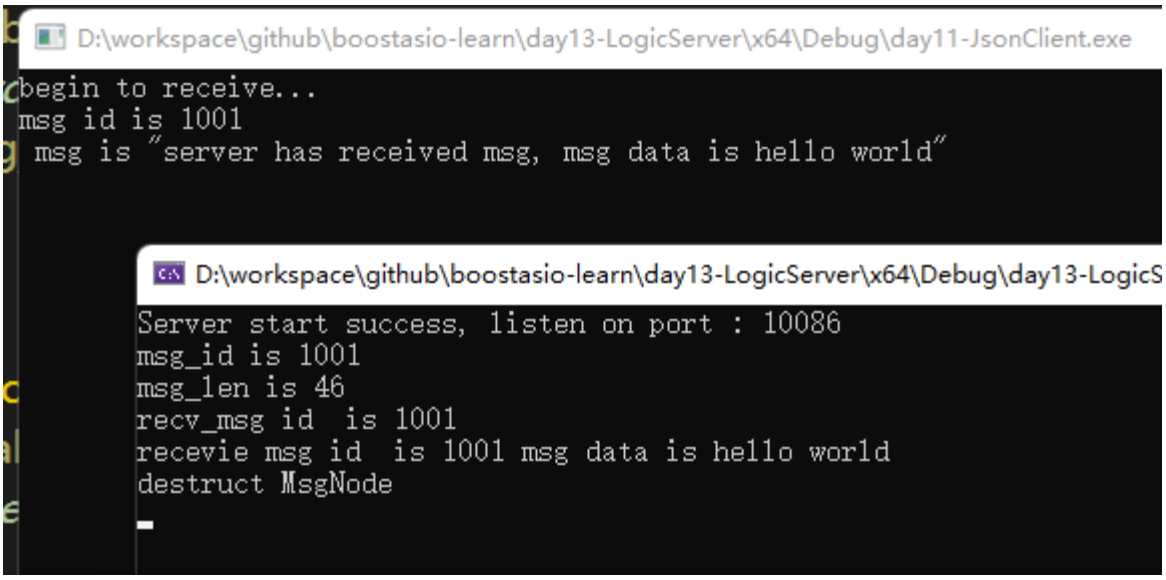
因为网络层收到消息后我们需要将消息投递给逻辑队列进行处理，那么LogicSystem就要封装一个投递函数

```
1. void LogicSystem::PostMsgToQue(shared_ptr < LogicNode> msg) {
2.     std::unique_lock<std::mutex> unique_lk(_mutex);
3.     _msg_que.push(msg);
4.     //由0变为1则发送通知信号
5.     if (_msg_que.size() == 1) {
6.         _consume.notify_one();
7.     }
8. }
```

在Session收到数据时这样调用

```
1. LogicSystem::GetInstance()->PostMsgToQue(make_shared<LogicNode>(shared_from_this(), _
   recv_msg_node));
```

再次启动服务器，编译启动，和之前一样可以看到数据收发正常。如下图:



总结

本文实现了服务器的逻辑类，包括并发控制等手段。

视频连接https://space.bilibili.com/271469206/channel/collectiondetail?sid=313101 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=313101)

源码链接https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(3272) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) 喜欢(590) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论

没有更多评论了,点击收起 ^ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。
京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)